Jean Biche

Informatique

I.

Q1. Oui / Non / Non

Q2. Soit $(s_0, s_1, \dots, s_m)$ un cycle eulérien avec $s_0 = s_m$.

Alors pour tout $s \in S$, le degré de s est deux fois le nombre d'occurrences de s dans le cycle, car chaque occurrence correspond à deux arêtes (une avant et une après).

Donc <u>$\deg(s)$ est pair.</u>

Q3.
```
bool is_eulerian(){
    int d=0;
    for (int i=0; i<N; i++){
        for (int j=0; j<N; j++){
            if (graph[i][j] != NULL){
                d++;
            }
        }
    }
    if (d%2 != 0) return false;
}
return true; }
```

Q4. Soit $v_0 \in S$. Comme G est connexe et $N \geq 2$, $\deg(v_0) \geq 1$.

Comme $\deg(v_0)$ est pair, $\deg(v_0) \geq 2$.

On pose v_1 un voisin de v_0. Pour $i \geq 1$, on pose v_{i+1} un voisin de v_i différent de v_{i-1}.

Un tel sommet existe, car comme pour v_0, $\deg(v_i) \geq 2$.

Par le principe des tiroirs, il existe $0 \leq i < j \leq N$ tels que $v_i = v_j$.

Le chemin $\underline{(v_i, v_{i+1}, \dots, v_j)}$ est bien un cycle.

Q5.

```
edge * round_trip (int start){
    edge * e = any_edge_from (start);
    while (e.dst != start) {
        connect (e, any_edge_from(e.dst));
        e = e.next;
    }
    return e;
}
```

Q6.

```
edge* find_vertex_with_edges (edge * c){
    edge * cur = c;
    while (cur != c) {
        if (has_edges (cur.src)) {
            return cur;
        }
```

```
        cur = cur.next;
    }
    return NULL;
}
```

Q7

```
edge* eulerian_cycle(int start){
    edge* cycle = NULL;
    while (true){
        edge* e = round_trip(start);
        join(e, cycle);
        e = find_vertex_with_edges(cycle);
        if (e != NULL)
            start = e.src;
        else
            return cycle;
    }
```

Q8.

round_trip : en $O(N)$ car le cycle est de taille $\leq N$

find_vertex_with_edges : en $O(E)$ car il faut parcourir tout le cycle.

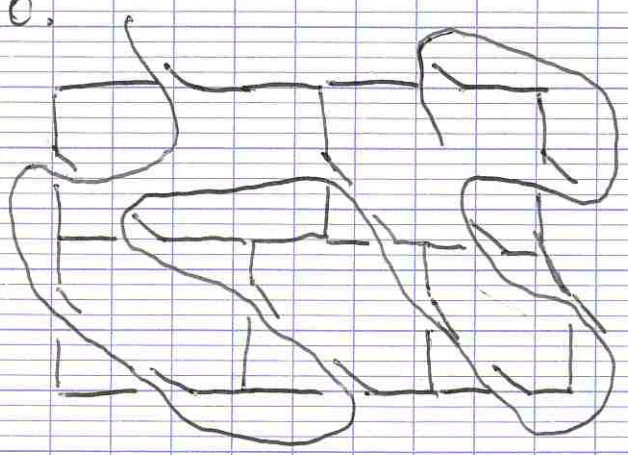
eulerian_cycle : la boucle est de taille $\leq E$, donc $O(E \times (E+N))$

On peut faire $O(N+E)$ avec un parcours de graphe.

Q9. Sens direct : comme Q2, sauf pour les extrémités.

Sens réciproque : comme Hierholzer en partant d'un sommet de degré impair.

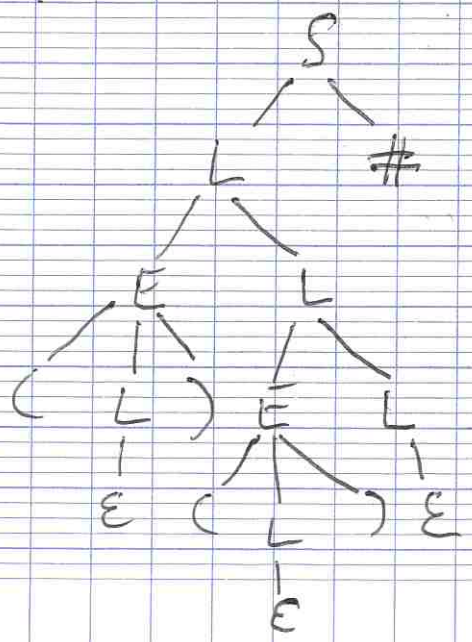
Q10.



Oui.

II.

Q11. Pour $n = 5$, le mot $()()\#$ est engendré par :



Jean Biche

Q12.

```
let parseL = function
    | (Rpar | Eof) -> Epsilon
    | l -> parseE (parseL l)

let parseE = function
    | Sym -> Sym
    | Rpar :: l ->
        let q = parseL l in
        List.tl q   (1)
    | _ -> raise SyntaxError
```

Je suppose qu'il est défini

(1) Pour enlever la parenthèse fermante.

Q13.

```
let accepte l =
    parseS l = []
```

Q14.

Il n'y a que L, car c'est le seul avec une règle $L \to \varepsilon$

Q15.

Le nombre de X tels que NUL(X) vaut false est un invariant de boucle, donc l'algorithme termine.

Q16. Je ne suis pas sûr de comprendre la question... C'est par définition de l'algorithme ?

J'essaie quand même :

À l'étape 2, s'il y a une règle $X \to \varepsilon$, on pose NUL(X) à true, donc l'algorithme trouve bien la bonne valeur de NUL(X).

Q17.

	S	L	E
PREMIERS	L	ε, E	symb, (
SUIVANTS	#	L	L

Q18. On a juste remplacé $L \to EL$ par $L \to LE$.

Vu que L et E engendrent toujours les mêmes langages, que $L = E^*$, le sens de concaténation ne change pas les mots obtenus.

Q19

	symb	(	)	#
S'	L#	L'#		L'#
L'	L'E'	L'E'	ε	ε
E'	symb	(L')		

On peut faire l'analyse comme avant, en inversant l'ordre des appels à parseL et parseE.

III.

Q20.

```
let rec check n = function
  | True -> true
  | False -> false
  | If( i, g, d) ->
      let rec aux = function
          | If (j, g, d) when j < i -> false
          | If (_, g, d) -> aux g && aux d
          | _ -> true
      in
      aux g && aux d && i < n
```

Q21.

If(0, If(1, true, false), If(2, false, true))

on inverse car 0 vaut false.

Q22. On a n variables, mais il n'est pas précisé si elles sont entre 0 et n-1, donc il faut renumérotter...

```
let renumerote n f =
  let cur = ref 0 in
  let dico = Hashtbl.create () in
```

```
let rec aux = function
    | True -> True
    | False -> False
    | If(i, g, d) ->
        if !(Hashtbl.mem dico i) then
            Hashtbl.add dico i cur;
            incr cur;
        If(Hashtbl.find dico i, aux g, aux d)
in
aux f;;
```

Maintenant on peut faire du backtracking.

```
let decide n f =
    let tab = Array.make n true in
    let rec aux = function
        | True -> Tautology
        | False -> Refutation of tab
        | If(i, g, d) ->
            if aux g = Tautology then
                tab.(i) <- false;
                aux d
            else Refutation of tab
in
aux (renumeroter n f)
```

Jean Biche

Q23.

```
let rec mk_not = function
  | True -> False
  | False -> True
  | If (i, g, d) -> If (i, mk_not d, mk_not g)
```

on inverse car on prend la négation.

Q24.

```
let rec mk_or f g = match f, g with
  | True, g -> True
  | False, g -> False
  | If(i, fg, fd), If(i, gg, gd) ->
      If(i, mk_or fg gg, mk_or fd gd)
  | _ -> mk_or g f
```

La complexité est linéaire.

Q25. On veut $\neg(F_1 \wedge F_2 \wedge F_3 \wedge F_4 \wedge F_5 \wedge F_6)$.
On utilise de Morgan.